23. Combined Decimation and Interpolation

Simulate a multirate system using decimation and interpolation and analyze the resulting

sampling rate conversion. (25) (BL4) (WK4)

Given Data:

● Decimation Factor = 3

● Interpolation Factor = 2

● Input Sampling Rate = 18 kHz

Tasks:

1. Design an anti-alias filter.

2. Perform decimation.

3. Implement interpolation.

4. Analyze the output spectrum.

5. Determine the final sampling rate.

6. Compare input and output signals.

Aim

To simulate a multirate system using decimation and interpolation, perform sampling rate conversion, and analyze the output signal and spectrum.

Introduction

A multirate system changes the sampling rate of a digital signal using decimation (reducing the sampling rate) and interpolation (increasing the sampling rate). An anti-aliasing filter is applied before decimation to prevent aliasing. In this experiment, the input sampling rate of 18 kHz is first decimated by 3 and then interpolated by 2, resulting in a final sampling rate of 12 kHz.

Code

```
% Combined Decimation and Interpolation
% Fs = 18 kHz, M = 3, L = 2
```

```
clc;
clear;
close all;

%% Input Signal
Fs = 18000;          % Input Sampling Rate
t = 0:1/Fs:0.01;     % 10 ms duration

% Signal containing frequencies below cutoff
x = sin(2*pi*1000*t) + 0.5*sin(2*pi*2000*t);

%% Anti-Alias Low Pass Filter
fc = 3000;                    % Cutoff Frequency
Wn = fc/(Fs/2);                 % Normalized Cutoff
N = 50;                       % Filter Order

b = fir1(N,Wn,'low');             % FIR LPF
x_filt = filter(b,1,x);

%% Decimation (M = 3)
M = 3;
x_dec = downsample(x_filt,M);
Fs_dec = Fs/M;

%% Interpolation (L = 2)
L = 2;
x_int = upsample(x_dec,L);

% Reconstruction Filter
```

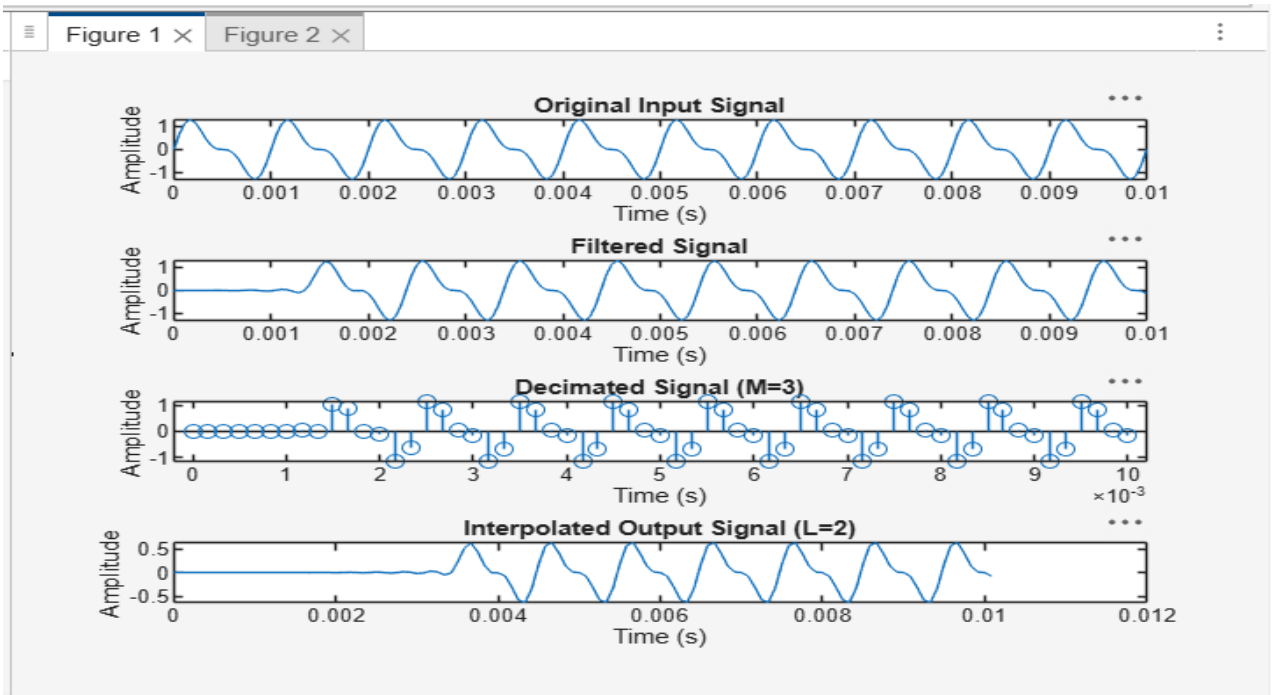
```
Fs_out = Fs_dec*L;
fc2 = Fs_dec/2;

Wn2 = fc2/(Fs_out/2);
b2 = fir1(N,Wn2,'low');

x_out = filter(b2,1,x_int);

%% Final Sampling Rate
fprintf('Input Sampling Rate = %d Hz\n',Fs);
fprintf('After Decimation = %d Hz\n',Fs_dec);
fprintf('Final Output Sampling Rate = %d Hz\n',Fs_out);

%% Time Domain Plots
figure;

subplot(4,1,1);
plot(t,x);
title('Original Input Signal');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(4,1,2);
plot((0:length(x_filt)-1)/Fs,x_filt);
title('Filtered Signal');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(4,1,3);
stem((0:length(x_dec)-1)/Fs_dec,x_dec);
```

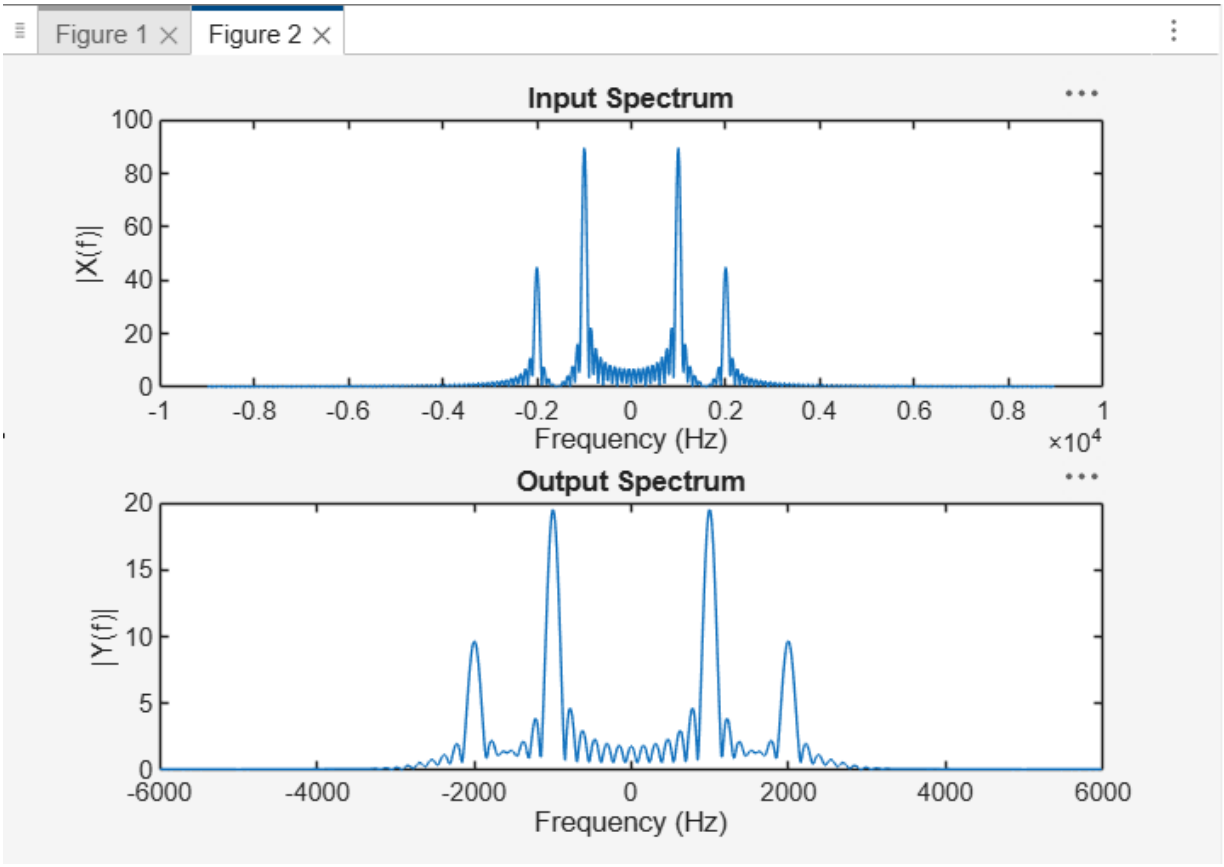
```
title('Decimated Signal (M=3)');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(4,1,4);
plot((0:length(x_out)-1)/Fs_out,x_out);
title('Interpolated Output Signal (L=2)');
xlabel('Time (s)');
ylabel('Amplitude');

%% Spectrum Analysis
figure;

subplot(2,1,1);
Nfft = 1024;
f1 = Fs*(-Nfft/2:Nfft/2-1)/Nfft;
X = fftshift(abs(fft(x,Nfft)));
plot(f1,X);
title('Input Spectrum');
xlabel('Frequency (Hz)');
ylabel('|X(f)|');

subplot(2,1,2);
f2 = Fs_out*(-Nfft/2:Nfft/2-1)/Nfft;
Y = fftshift(abs(fft(x_out,Nfft)));
plot(f2,Y);
title('Output Spectrum');
xlabel('Frequency (Hz)');
ylabel('|Y(f)|');
```

Result

- Anti-alias filter successfully prevented aliasing.
- The signal was decimated by 3 and interpolated by 2.
- The final sampling rate obtained was 12 kHz.
- The output spectrum showed reduced aliasing and proper sampling rate conversion.
- The input and output signals were similar, with slight changes due to filtering and resampling.

Output






```
Command Window

Input Sampling Rate = 18000 Hz
After Decimation = 6000 Hz
Final Output Sampling Rate = 12000 Hz
>>
```